SDU
University of Southern Denmark

Design Goals for End-User Development of Robot-Assisted Physical Training Activities

A Participatory Design Study

Gutierrez, Jose Pablo De La Rosa; Silva, Thiago Rocha; Dittrich, Yvonne; Sørensen, Anders Stengaard

Published in:
Proceedings of the ACM on Human-Computer Interaction

DOI:
10.1145/3664632

Publication date:
2024

Document version:
Final published version

Document license:
CC BY

Citation for pulished version (APA):
Gutierrez, J. P. D. L. R., Silva, T. R., Dittrich, Y., & Sørensen, A. S. (2024). Design Goals for End-User Development of Robot-Assisted Physical Training Activities: A Participatory Design Study. *Proceedings of the ACM on Human-Computer Interaction*, *8*(EICS), Article 258. https://doi.org/10.1145/3664632

Go to publication entry in University of Southern Denmark's Research Portal

Terms of use
This work is brought to you by the University of Southern Denmark.
Unless otherwise specified it has been shared according to the terms for self-archiving.
If no other license is stated, these terms apply:

- You may download this work for personal use only.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying this open access version

If you believe that this document breaches copyright please contact us providing details and we will investigate your claim.
Please direct all enquiries to puresupport@bib.sdu.dk

Download date: 07. Nov. 2026



Design Goals for End-User Development of Robot-Assisted Physical Training Activities: A Participatory Design Study

JOSE PABLO DE LA ROSA GUTIERREZ, University of Southern Denmark, Denmark
THIAGO ROCHA SILVA, University of Southern Denmark, Denmark
YVONNE DITTRICH, IT University of Copenhagen, Denmark
ANDERS STENGAARD SØRENSEN, University of Southern Denmark, Denmark

Programming robots presents significant challenges, including high costs, extensive time commitments and steep learning curves, particularly for individuals lacking technical background in engineering. These barriers have been partially mitigated by the emergence of end-user development methodologies. Yet existing approaches often fall short in equipping users with the necessary software engineering competencies to develop comprehensive robot behaviors or to effectively maintain and re-purpose their creations. In this paper, we introduce a novel end-user development approach designed to empower physical therapists to independently specify robot-assisted physical training exercises, eliminating the need for robotics experts' intervention. Our approach is based on a set of design goals obtained through a participatory design study with experts in the field. It utilizes a textual domain-specific language (DSL) that enables users to define expected robot behaviors through Behaviour-Driven Development (BDD) scenarios. This paper discusses key themes, design objectives, and the evolution of requirements that emerged from an evaluative workshop.

CCS Concepts: • **Human-centered computing** → **HCI design and evaluation methods**; **Graphical user interfaces**; *Empirical studies in HCI*; • **Computer systems organization** → *External interfaces for robotics.*

Additional Key Words and Phrases: Low-Code Development, Robot Programming, End-User Development, Behaviour-Driven Development (BDD), Domain-Specific Languages (DSLs).

ACM Reference Format:
Jose Pablo De La Rosa Gutierrez, Thiago Rocha Silva, Yvonne Dittrich, and Anders Stengaard Sørensen. 2024. Design Goals for End-User Development of Robot-Assisted Physical Training Activities: A Participatory Design Study. *Proc. ACM Hum.-Comput. Interact.* 8, EICS, Article 258 (June 2024), 31 pages. https://doi.org/10.1145/3664632

1 INTRODUCTION

Robotics have introduced the possibility of augmenting traditional physiotherapeutic interventions with technologies that are either monitored or controlled by computers. These innovations range from passive wearable monitors to active computerized body-unloading technologies (e.g., [1, 6]), exoskeletons (e.g., [3, 9]), and cable-based training devices (e.g., [39], [68]). Such devices can vary in their participation in training activities from merely blending into the user's environment as unobtrusive body-unloading platforms, to active participatory roles supporting or challenging complex movements. The way in which these technologies can be used largely depends on the

Authors' Contact Information: Jose Pablo De La Rosa Gutierrez, jode@mmmi.sdu.dk, The Maersk Mc-Kinney Moller Institute, University of Southern Denmark, Odense, Denmark; Thiago Rocha Silva, thiago@mmmi.sdu.dk, The Maersk Mc-Kinney Moller Institute, University of Southern Denmark, Odense, Denmark; Yvonne Dittrich, ydi@itu.dk, Computer Science Department, IT University of Copenhagen, Copenhagen, Denmark; Anders Stengaard Sørensen, anns@mmmi.sdu.dk, The Maersk Mc-Kinney Moller Institute, University of Southern Denmark, Odense, Denmark.



This work is licensed under a Creative Commons Attribution International 4.0 License.
© 2024 Copyright held by the owner/author(s).
ACM 2573-0142/2024/6-ART258
https://doi.org/10.1145/3664632

patient's needs, with different types of control modes appropriate for each phase of rehabilitation [14]. However, current clinical device configuration systems require users to align with the abstractions and usage catalogs provided by manufacturers. Systems such as lifts or weight relief devices are typically controlled through a physical interface, allowing for on-the-fly adjustments with a limited range of controls that a therapist can modify during a session, such as support magnitude or the maximum stretch level of a rope to prevent falls. The use of these tools in clinical settings is confined to a passive mode, where the device behavior remains static, regardless of the patient's response, without allowing for the design of interactions that emerge from the patient's contributions. Exoskeletons, on the other hand, offer a higher level of granularity and an extensive catalog of configurations that can be set in advance through wizard-based user interfaces on computers to adapt specific dynamic behaviors according to the physical characteristics of users. Nevertheless, the range of movements allowed by these devices is still limited to the use cases available in the wizard, making difficult the implementation of divergent intervention trends (e.g., error augmentation [51]).

Within the expanding body of literature focused on enhancing flexibility in sports and therapy domains, prior studies by Kosar et al. [42], Tetteroo et al. [70] and Payne et al. [59] demonstrate innovative solutions that empower end-users to customize programs for bodily-interactive applications, drawing on their creativity and domain-specific knowledge. However, these methods have not yet explored the shaping of forced-based interactions in clinical contexts, where therapists must adjust force signals that a control system applies to a human based on their movements. In this work, we propose a solution that enables therapists to shape these interactions using a straightforward, cable-based training robot, setting the stage for potentially extending these innovations to more complex robotic training systems.

Cable-based therapy devices offer a cost-effective, simple method for physical exertion by loading or unloading a participant's weight. A single-degree-of-freedom cable-based robot can emulate the gravity and inertia offered by weights in conventional exercise machines or the buoyancy of semi-submerged water training. In these cases, the robot pulls the wire as a continuous function of time and motion to mimic the participant's expectations of natural mechanical dynamics, such as Newton's and Archimedes' laws. While physiologically effective, training based on continuous functions may lack motivational elements that occur when training participants interact with objects, devices, or co-participants that show non-continuous behaviors such as a bouncing ball or a tug-a-war with human peers.

While conventional robot programming paradigms are well suited for programming non-continuous motions and forces, only some training experts have the proper education, experience, or interest in conventional robot programming to develop them. Furthermore, current commercial technologies feature closed systems that do not allow for significant changes to the devices beyond specific parameters defined by the manufacturer, resulting in a control strategy and, consequently, a training modality that lacks flexibility. This creates a gap between the potential benefits of robotic assistance in therapy and the practical ability of healthcare professionals to deploy these technologies in a manner that truly enhances patient care by adapting to the individual preferences of patients and practitioners. In order to allow training experts to fully benefit from training robots, a new programming paradigm is necessary - a paradigm where training professionals can directly utilize their knowledge and experience to create robot programs that enact their thoughts on training.

In this paper, we report on a participatory design study conducted with experts in the field to uncover the design goals of a end-user development environment. Such environment is intended to support them in programming the desired behaviour of training devices. These design goals have emerged from qualitative analysis of formative interviews with physical therapists currently

working with robot-assisted rehabilitation. As a result, we devised a domain-specific language (DSL) that relies on the scenario-based specification provided by Behaviour-Driven Development (BDD) scenarios [56]. Our approach was designed for the RoboTrainer [68], a proof-of-concept device which consists of a one-degree of freedom low-cost rope pulling robot that prioritizes simplicity, versatility, and cost-effectiveness as fundamental design principles (Fig. 1). This approach ensures that we can obtain a detailed case study of bodily human-robot physical interaction without compromising its physiological effectiveness [66]. Such deployment highlights the potential transferability of our language within the wider spectrum of cable-based training robotics (e.g., [45, 50, 64]).

The core inquiry of this research centers around how a DSL based on BDD could allow domain experts in physical therapy to overcome the learning barriers experienced when mapping their domain requirements (specified in their own vocabulary) to the set of actions and conditions implementable by a training robot. This requires, on the one hand, a deep understanding of the user and its practices while, on the other, defining a programming language that facilitates the process of authoring training sessions assisted by the robot, along with the software and hardware architecture that supports the user's work in a said environment.

This paper is structured as follows: **Section 2** introduces the BDD method and provides an overview of related work within end-user robot programming techniques and guidelines. **Section 3** presents our qualitative analysis methodology and the key-themes identified from the formative interviews conducted with physiotherapists. **Section 4** introduces the design goals identified from the themes. **Section 5** presents our proposed domain-specific language based on the identified goals. **Section 6** describes in detail the implementation's configuration and components. **Section 7** illustrates the workflow of our tool based on a common cable-based resistance training activity. **Section 8** presents an evaluation workshop performed with a domain expert, and further refinement opportunities identified from the resulting data. **Section 9** discusses some implications from our work on a broader contexts. **Section 10** presents the limitations of the solution. Finally, **Section 11** provides the conclusions of the paper and future work to be developed.

2 BACKGROUND AND RELATED WORK

2.1 Human-Robot Interaction Guidelines for Computer Interfaces

When designing programming systems for robots, several aspects are crucial for providing an integrated and user-friendly experience. As discussed in previous research [11], the following design strategies have been identified in the literature:

- Enabling re-usability of program elements, fostering a modular design that allows flexible combinations of elements and the creation of general-purpose components (e.g., [49]).
- Merging with the physical environment and activities, leveraging augmented reality (AR), natural language-based interfaces, and tangible programming features to provide a clear understanding of scenarios and expected results (e.g., [44, 67]).
- Enhancing programming transparency through real-time status displays, clear command sequences, programming by demonstration, and process mapping techniques, enabling users to develop accurate mental models of the robot's capabilities and limitations (e.g., [32]).
- Moderating abstraction levels by providing primitive or semantic-level commands and modular structures to address the challenges of abstract programming concepts such as loops and logical conditions (e.g., [43]).
- Providing assistance through error prevention, detection, and handling mechanisms, as well as automation features, to reduce complexity and guide users in real-time during the program development process (e.g., [12, 21]).

Along with participatory design studies, these strategies help us in identifying promising development paths for our solution, ensuring an alignment with end-user needs as well as the above listed guidelines.

2.2 End-User Development and End-User Robot Programming

End-User Development (EUD) [48, 58] and End-User Software Engineering (EUSE) [40] have been studied for many years in software engineering. 'End-User Development can be defined as a set of methods, techniques, and tools that allow users of software systems, who are acting as non-professional software developers, at some point to create, modify or extend a software artefact' [48].

The core problem of EUD is to represent computational functionality in a way that allows domain experts to comprehend the adaptation possibilities and relate them to there specific needs. In academia and industry, a number of approaches and notations for EUD have been explored [13]. To our knowledge there is no systematic guide for the selection of an approach based on EUD target software or EUD domain. We therefore here discuss the approaches most relevant for our case: programming by demonstration (PBD), visual programming (VP), and trigger-action programming (TAP).

Programming by demonstration (PBD), despite offering a straightforward approach for defining repetitive industrial tasks [33], can constrain the versatility of interactive systems, as demonstrations are hard to generalize, and lack re-usability throughout different contexts. Visual programming (VP), on the other hand, addresses these challenges by allowing procedural authoring using different approaches such as blocks (e.g., [16, 34]) and graphs (e.g., [28, 60]). Nevertheless, while these methods enhance re-usability and simplicity, their use generally relies on a basic understanding of the underlying code structure, which requires an investment of learning and practicing time to take full advantage of the tools.

Finally, trigger-action programming (TAP) stands as a popular alternative for simplifying automation tasks by mapping events or conditions (triggers) to particular responses available on the system (actions) [57]. This facilitates the development of tasks without the need for algorithmic skills, which has proven successful in applications for Internet of Things (IoT) (e.g., [15]), augmented reality (e.g., [54]), and social robotics (e.g., [47]), among others.

Besides suitable representations for EUD, the correctness of programs and adaptations has been on of the key hinders for users to engage in EUD independently [25]. The survey article by Ko et al. [40] likewise highlights the need for systematic quality assurance. Researchers explored testing [18] and formal methods [17] for EUD. Another promising approach is to understand and design EUD notations as Domain Specific Languages (DSLs). From a programming language point of view, any EUD notation can be regarded as a DSL. DSLs present the opportunity to provide a level of abstraction appropriate to the user through solutions of limited expressiveness tailored for fulfilling their particular domain needs [27]. Furthermore, the exploration of traditional DSLs has been significantly advanced through their integration with prevalent VP environments. This simplifies discoverability while encouraging the use of natural language as evidenced by related research [53, 71]. Further, using the research on domain specific languages to inform the design and implementation of an EUD notations and interface allows to use programming language technology to prevent certain kinds of errors systematically. This is especially relevant when it comes to rehabilitative use of robots, where unexpected behaviour can scare or even harm patients and therapists or might destroy the robot itself. Though this article addresses the design of the EUD notation for physiotherapists, the possibility to support the correctness of the programs has informed the choice of implementation approach as well.

2.3 Behaviour-Driven Development

Behaviour-Driven Development (BDD) was originally proposed in 2006 [55] as an effective method to specify software requirements and their acceptance criteria with the aim of reducing the communication gap between subject-matter experts and software developers. BDD practitioners use conversations around concrete examples of system behaviour [10] to help understand how features will provide value to the business. The method encourages stakeholders to collaborate more closely by enabling them to express requirements in a more testable way, using a format that both the development team and the business stakeholders can easily understand.

For that, BDD relies on scenarios [56] written in a semi-structured natural language format based on the Gherkin syntax [7]. Each scenario (see below) includes a title and three key clauses: "Given", that provides the context in which the scenario executes; "When", that identifies the action being performed in the scenario; and "Then", that presents the expected system outcomes. To provide several contexts, actions, and/or outcomes, each one of these *steps* may contain an "And" statement. In our approach, BDD scenarios are seen as complete state transitions on a state machine.

```
Scenario: <title>
Given [context]
And [some more context]...
When [event]
And [another event]...
Then [outcome]
And [another outcome]...
```

BDD is similar to TAP approach described above: BDDs are based on a clear framework of events ("When") and expected outcomes ("Then"). Nevertheless, the "Given" statements in BDD provide an additional level of granularity, enabling the user to specify the particular context in which the event must be triggered. Furthermore, the context-event-outcome structure of BDD scenarios allow for a complete representation of the state transitions on a state machine, making it possible to provide reliability guarantees of the specification. The completeness of BDD scenarios is also particularly relevant within the scope of robot-assisted physical therapy, where the behavior of the device depends on particular phases of the therapy session or individual phases of an exercise.

The Gherkin syntax, however, does not limit what can be specified within each clause. Prior work has proposed a domain-specific language (DSL) to support the writing of consistent BDD scenarios [62, 65] and constrain the specification to known constructs. The use of a DSL for writing BDD scenarios allows for systematic parsing of the steps and the execution of scenarios through automated testing frameworks (e.g., [4, 5, 8]). In this work, we adapt this DSL to allow end-user developers to specify and execute robot behaviours while constraining the resulting program to avoid dangerous robot movements.

3 FORMATIVE INTERVIEWS WITH EXPERTS

3.1 Methods and Procedure

To ground our research in the context of physiotherapists and their real-world practices with robot-assisted training technologies, we gathered qualitative data from semi-structured interviews at a local therapy clinic, known for its widespread adoption of cutting-edge rehabilitation technologies (e.g., [1, 3, 6, 9]). Five participants were interviewed, all identified as professional therapists working with diverse weight-relief technologies and exoskeletons on a regular basis. Our interviews focused particularly on their use of the body-unloading technologies they worked with such as LiteGait [1] and Ergo Trainer [6], as these have similarities with our proof-of-concept case.

A categorization of the empirical data gathered from the interviews and field notes was performed by drawing methods from grounded theory [20]. This involved an iterative process of line-by-line open coding, synthesis through focused coding, and categorized coding providing the key themes (T) discussed below.

3.2 Participants' Profile

The interviews commenced with participants introducing themselves and offering their essential demographic details. (Table 1). Despite focusing on treating patients with neurological injuries, like those recovering from strokes, they showed significant differences in experience, education, and specialization in treatment approaches. Educationally, the group varied from those holding bachelor's degrees to others with advanced master's qualifications, some specializing further in fields like bio-mechanics. In terms of treatment specialization, their practices cover a broad scope, from focusing on specific rehabilitation technologies and equipment for mobility enhancement, to developing and teaching new treatment protocols.

Table 1. Demographic data gathered from the participants interviewed: Age, gender, experience working as physiotherapist(years), experience working with robots(years), and academic level.

	P1	**P2**	**P3**	**P4**	**P5**
Age	43	35	41	28	46
Gender	F	F	F	M	M
Exp. as Physiotherapist	17	11	15	2.5	8
Exp. with Robots	10	10	15	2	7
Academic Level	Bachelor	Bachelor	Bachelor	Bachelor	Master

3.3 Findings

3.3.1 ***T1 - Preserving and Sharing Knowledge.*** The diversity of profiles among the participants reflects the wide range of expertise found in clinical settings, leading to different knowledge bases and skills among therapists. In practical terms, this diversity results in therapists adopting various styles in therapy sessions. Their choice from the practical resources available in the clinic and the way of using them is guided by previous learning and experiences and aimed at applying such knowledge in providing benefits to patients. As an example, in **C1**, a participant discusses the selection of a particular intervention focusing on the elements that align closely with their background. This comment also highlights the inherent creative freedom therapists have in choosing their preferred interventions aiming to achieve the best possible outcomes for their patients. Therefore, the integration of the clinic's available resources into their practices is partly determined by their coherence with the competences developed through their background, and partly by the needs from the patient (**C2**).

C1 *"I try to select what is the main problem. What do I need to examine more? And what do I have in my background? (…) So, it's up to each therapist to find out what do we have in the back to put up? But it [also] depends on the patient. What is their goal and which are their problems? We have newly educated physiotherapists, and we have the old ones, and we have gone through a lot of different courses (…). So, it's up to each therapist what we use."*

C2 *"Not all of us have the same skills. Not every patient has the same therapies. Because we don't have the same skills all around. For instance, this high intensity walking. All of us should know it. But not all of us are as quick as others taking the new skills or equipment into practice."*

Reflecting the core role of knowledge in their practice, participants consistently highlighted the need for preserving their knowledge base and collaborating with other therapists to enrich and expand their competencies. In terms of preservation, **C3** reflects on the relevance of critically evaluating what knowledge to retain and how to adapt it for innovations in their field, preventing the cyclical loss and rediscovery of effective interventions. In terms of growth through collaboration, **C4** deepens into how the exchange of ideas and experiences allows them to refine their skills, learn new techniques, and stay updated on best practices. In consequence, deploying technology that leverages on therapists' innovative ideas for patient care is vital, encouraging collaboration and adaptable use of diverse therapeutic practices.

C3 *"You must still have the background into the future. How do you carry the original way into that future without just throwing stuff away? Because lots of wisdom has been lost because it hadn't gained the evidence and then simply was just discarded. Stretching techniques, all kind of stuff. And then you reinvented like 30 years later. So, it's really important to [reflect], what do you bring? What do you keep? "*

C4 *"Once a week we meet other physios and we can discuss a certain patient (...). We'll help each other finding the right way to treat the patient. And so we try to develop each other as physiotherapist in this group and also in the house."*

3.3.2 ***T2 - Adopting Robots in Therapy.*** All participants confirmed using weight-relief technologies for their therapies on a regular basis. During the interview, they actively commented about the difficulties experienced before the adoption of such technologies. These challenges included the need to physically support patients who lacked the ability to stand independently, often requiring additional assistance from two therapists, and the considerable physical and cognitive demands placed on them to ensure patient safety and prevent falls. Consequently, the range of activities performed while standing was restricted to static movements on standing tables (**C5**). Additionally, participants perceived patient progress in sessions as slow and cautious, attributed to the necessity of fulfilling numerous prerequisites before walking, all driven by the imperative to prevent falls in the clinic (**C6**).

C5 *"We couldn't walk with them. We could stand with them, with a lot of support. And we had a standing table with the harness to the hip and some stability to the arms. And there were two therapists standing on both sides. And they could stand up and that's it."*
C6 *"They had to be more careful, you could say. You had to be able to really carry your own weight without any risk of falling. And that makes it a much more careful regime. Because you needed this checklist of what your body should be able to do (before walking)."*

Following the introduction of unloading devices, the responsibility of supporting the patient and preventing falls largely shifts to such robotic systems. Patients are transferred to the training area and placed in a harness that is attached to the device's end effector, often a solid crossbar of variable height ([1]) or an actuated cable system ([6, 68]), which supports a portion of the patient's weight, allowing for free movement during standing activities either on a treadmill or free ground. Furthermore, fall prevention is ensured through safety features designed to statically suspend the patient's full weight if falling below certain thresholds. This permits a greater portion of training sessions to be dedicated to dynamic, functional activities, such as walking. In discussions **C7** and **C8**, participants shared how this new approach helps patients become more independent during sessions. They can walk freely and practice their skills through trial and error, thanks to the robots acting as a safety net.

C7 *"Before we were very focused on symmetry and function. And, you know, weight, transference and making the legs strong enough to walk. And the couple of last years we've been focusing more on walking, just walking. Also still doing some of the old stuff. But more and more letting the patients walk. The way they can."*
C8 *"Evidence and data from studies that have been going on the last ten years says that it's good for the patient to get the feedback from themselves. And it's good for the patients to make some errors. And now we can do it because we have the equipment to let them make their errors."*

However, despite the trending focus towards these walking tasks, some therapists still wish to retain the opportunity to blend classic styles of therapy with the new advantages of robotic support, thereby maintaining a balance between innovative interventions and traditional methods that promote strength and movement technique. For example, as detailed in **C9**, a participant with expertise in sports rehabilitation describes their approach to adapting the use of a cable-based weight-relief robot for heavy resistance training with patients' own body weight. This blend of practices leads to the identification of two primary roles for the weight-relief robot (Fig. 1):

(1) **Passive Unloading**: the robot is configured to seamlessly blend into the user's environment. It offers weight relief for individuals undertaking rehabilitation tasks, aiding post-stroke victims and patients with cerebral palsy. Notable examples include walking assistance, trampoline jumping, and wall climbing. These activities necessitate sophisticated software modeling of continuous force feedback from the robot, ensuring a smooth and transparent experience for the user in each task (e.g., support while walking [66]).
(2) **Active Assistance/Resistance**: The robot takes on a visible, participatory role in the training activity, providing users with opportunities to challenge their maximum force capabilities. Such activities may often involve cyclic tasks with variable discontinuous force responses (e.g., concentric-eccentric training [26]).

C9 *"All my resistance training is built around the demands of real movements. Mainly about gait and running. The idea is, with the [robot], you can take weight off. Often heavy resistance training is about putting weight on, and that's our mindset, right? You squat, you put on extra weight, and put on extra weight. So suddenly you have a patient who can't squat with their own weight. So, you take weight off."*

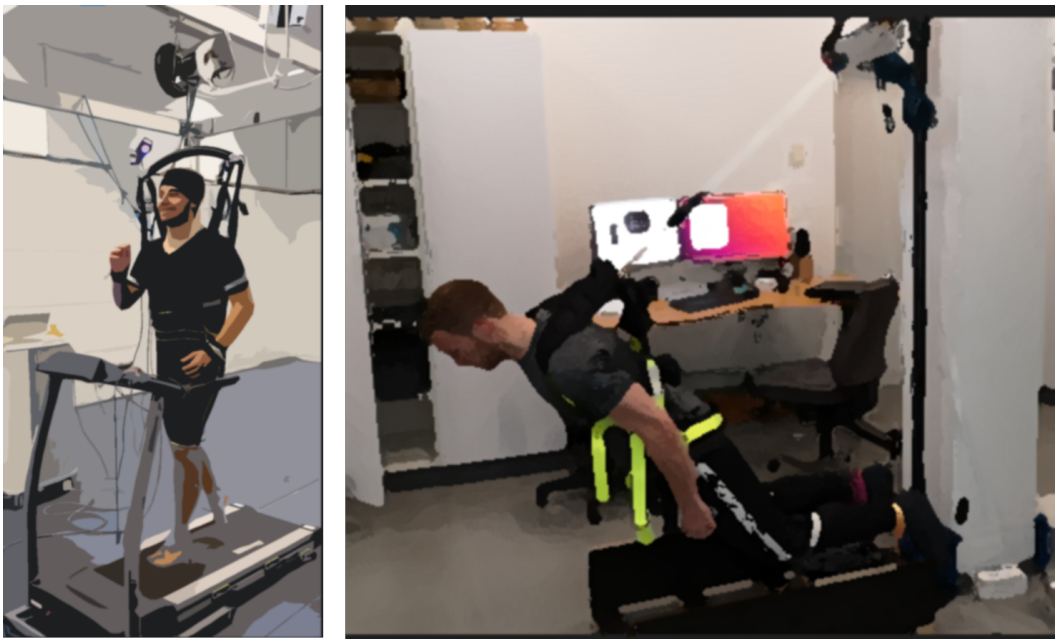


Fig. 1. Two different use cases are identified: Weight-relief for functional gait training (left) and granular support for specific motion patterns (right).

3.3.3 ***T3 - A Methodical, Data-Driven Progression***. Participants commonly acknowledged the importance of keeping their therapy plans current, with a frequent consensus on focusing activities on the patient's primary motor challenges. For example, **C10** offers an insight of the evolving strategies employed by one participant in a typical gait rehabilitation scenario. This approach involves progressively addressing individual gait components based on the continuous assessment of the patient's performance during treadmill walking sessions. The need to constantly track progress across sessions motivates the therapists to keep a detailed log of the settings and performance metrics shaping each activity. This ensures that training remains consistent at the start of each session, the tasks' level of difficulty is appropriately adjusted over time, and the exercises remain correctly focused towards relevant problems.

C10 *"A typical patient in the beginning has no balance, no swing, no stance, no propulsion. And I always start with the swing. I suspend them in the harness with body-weight support, and make them do a lot of swinging the leg. And I close my eyes for all the other stuff. And maybe in two days they learn how to do it themselves. So, when they have a more stable swing leg, we can start focusing more on the stance. And it can be a week or ten days before they are ready for that."*

Documenting the settings of each machine as part of the training logs has several advantages: functional for defining the behaviour of the machine, strategic for tracking patient progression and keeping an updated activity plan, and documentative for facilitating collaboration among healthcare providers. In **C11**, a participant exemplifies this point by comparing two relief systems in the clinic, highlighting the value of monitoring progress via equipment settings. The first system, a force control mechanism, provides precise relief in kilograms, while the second, a position control system, offers adjustable relief based on visual assessment. The preference for the first system is due to its precise progress tracking capability, since the relief quantity is defined in numerical, exact amounts of kilograms, while the second technology requires an intuitive observational setup that can hardly be persisted through sessions.

C11 *"The system to take off the weight of the patient is better in [tech1] than it is in [tech2]. [tech2] is fiiim (simulates a lift with a button). You don't know how much body weight you support the patient with. So it is hard to do the same training with the same amount because you cannot see how much support you have given to the patient (...). It's easier in the [tech1] to compare the training session. Also, if we change therapist, it would be easier for if [they] can see 50 percent or 50 kilos."*

3.3.4 ***T4 - Key Focus Points During Therapy.*** We asked therapists how focusing on maximizing patients' independent walking during sessions influences their role. The consensus among participants is that they act as facilitators, shaping environments that offer relevant experiences with an appropriate level of challenge to encourage motor learning. This facilitation is achieved through two primary tools: (i) monitoring and controlling the patient's vital signs using high-intensity training methodologies, and (ii) quantifying and targeting specific error-by-repetition rates. Furthermore, the effort to control the walking task's challenge level is evolving into a quantifiable process. This development is in line with therapists' aim for methodical, data-driven progressions in patient care (T3), marking a significant shift towards more precise and measurable approaches to therapy.

C12 gives an overview of how the high-intensity training methodology is used to guarantee that the patient exercises at a high cardiovascular effort. The process involves continuously monitoring the patient´s heart rate, while performing actions on the environment to change the difficulty of the task to meet a particular intensity range. On the other hand, **C13** exemplifies on the tracking of challenges by observing the error rate of the patient walking. The therapist constantly monitors these parameters and uses their professional vision to enable variations in the environment that reduce the error rate of the desired challenge in the task (Fig. 2). These variations not only achieve changes in the difficulty of the workouts, but also help the patient to exercise a catalog of movements that allow gradual functional integration to their community (**C13**).

C12 *"We put on the harness and we select what sub-component of the walking we would like to train (…). They are walking most of the training time. Maybe they can start ten minutes walking or five. Then they are tired. So next time, maybe 12 minutes. And it's about training at 70 to 55 percent of max heart rate. So it has to be hard training. They have to walk, walk, walk, walk, walk. And the possibility is to make their exercise more difficult. In the start, they just walk forward. Maybe later on they get more weight. They can walk backwards, sidewalks over steps and difficult uphill."*

C13 *"In the beginning, assist or guide the leg. And then, as the patient can take over, you start making it more difficult. So they make errors. We induce errors. We don't want them to walk perfectly. So when they begin to walk symmetrically, you can swing the leg every time. That's a cue for me to think that I have to do something more difficult. So you could put a weight manchette on the leg, or you could make the treadmill go upwards. You could increase the speed, you could lower the bodyweight support, you could move the exercise out on the floor still with the harness. But doing something more difficult by going around some of the obstacle. And also just to limit your own assistance."*

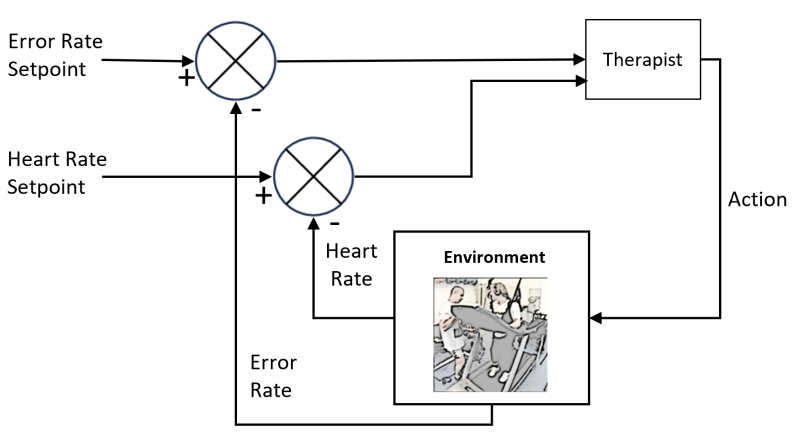


Fig. 2. Through actions changing the environment, the therapist aims at making tasks challenging for the patient. Common alternatives for determining the challenge of tasks rely on the heart rate and error-by-repetition rate.

Some variations mentioned by the participants include:

- **Robot variations**: Amount of weight relief, behavior of the end effector (static or dynamic).
- **Walking variations**: Walking speed, type of terrain (inclination, texture, steps, softness, indoors, outdoors), direction (sideways, backwards, front), pattern (*"walk like a ninja"*, *"walk like marching"*).
- **Task variations**: Simultaneous tasks performed (walk and talk, walk and stare at something), avoiding obstacles.
- **Additional limb weight**: Using manchettes or lifting dumbbells.
- **Therapist Intervention**: Amount of external help from the therapist.

Among the configurable features on the robot, therapists can adjust the amount of relief and the device's behavior, categorized as either dynamic or static. The dynamic setting correlates with

force-based control devices, allowing therapists to adjust the relief force applied to the patient via a remote control. Conversely, the static setting relies on position control, where the therapist determines the position of the robot's end effector and the smoothness of its response, essentially adjusting the control system's gain to modulate the action response.

3.3.5 ***T5 - Promoting Feedback.*** Participants emphasized the importance of providing patients with feedback on their activity performance as a key element in promoting motor learning. Yet, delivering consistent and clear feedback poses challenges, especially when therapists must manage multiple responsibilities simultaneously, risking the delivery of unreliable or ambiguous feedback (**C14**). A common approach involves encouraging patients to independently recognize correct movements through strategically placed cues across various sensory channels, such as visual bio-feedback from displays, mirrors reflecting the patient's posture, or auditory feedback from sounds made by the feet dragging on the treadmill (**C15**), instead of direct reinforcement from the therapist. Nonetheless, selecting appropriate feedback cues is complex and must be tailored to each patient's specific needs. This necessitates technologies that offer customizable access to state data from training devices, allowing therapists to use their professional experience for interpreting and adjusting feedback to different levels of abstraction. **C16** shows an example of different alternatives of feedback proposed by a participant to give knowledge of performance to the user while walking. It is of particular interest to observe how different degrees of abstraction are proposed for highlighting errors while walking: a mirror, a simple smiling face, or an animated man walking. The decision of a particular method is performed through an assessment of the patient.

C14 *"It's a problem because the patient has no other way of knowing without me saying it. And my feedback is unstable because sometimes I say "Good!". Maybe I didn't mean it, maybe it wasn't good. I say good anyways because I wanted to motivate this like that. When I tell them I give too many contradictory messages sometimes. Because I'm a human! I say the wrong things sometimes."*

C15 *"You can use the speed. When you turn up the speed, they have to manage to take faster steps and longer steps. So, you can use the environment. We try not to speak as much (...). They have to experience. Instead, if I say "Lift a leg. Put it down. Get forward. Stop". They will not learn as much as if they self-experience. I have to walk faster or lift my leg."*

C16 *"Sometimes they can use a mirror. But not everybody can do that (...). Maybe because of the eyesight, they have neglect. They have some injury to the eye muscles that can be double sided. A lot of things that they don't [perceive]. But if they have it on a screen that is close, they can see like a smiling face or they can see like a man going up and down. They can see: "Now I'm actually stretching my legs, so it goes up and I'm swinging my legs. So, it also goes up"."*

3.3.6 ***T6 - Time Negotiations.*** Therapists commonly agree that a typical session with a patient lasts one hour, a period during which they aim to optimize rehabilitation activities. However, they often face the challenge of balancing effective patient interaction with the preparation of technology. A key preparation activity is assisting the patient with weight-relief, involving fitting the harness, transferring them to the treadmill or another training surface, and adjusting the necessary relief, speed, or treadmill incline parameters. This preparation can take between 10 to 20 minutes of focused effort.

The approach to this task is adaptable, influenced by the therapist's experience, the patient's motor abilities, and the feedback system in use (**C17**). Dynamic relief systems, which directly attach the robot's end effector to the patient's harness, streamline the process by eliminating the need

for manual positioning with a remote control. In contrast, static systems, while providing stable support for heavier patients or those with limited mobility, require manual adjustment to position the end effector appropriately, adding steps to the preparation phase.

> **C17** *"I think the more that I have been doing it, the faster I'm getting at it. So now it takes no time to put on the harness. Often if it's a heavy patient I put the harness on in the bed when they're lying down and now it's like click, click, click in two minutes it's on. And then they are lifted to the wheelchair. And then we go down to the [tech1] and it's up and going so they can be ready in ten minutes on the treadmill."*

4 DESIGN GOALS

We used the themes and findings from the formative interviews presented in the previous section to generate design goals for end-user programming environments aimed at supporting physiotherapists on developing activities for cable-based physical therapy robots.

- **G1 Preservation of Therapist Knowledge**: The authoring platform should be adaptable to the knowledge base that therapists posses, seeking to use the training robot as a coherent channel to express their training style (T1).
- **G2 Flexibility for Divergent Therapy Practices**: The platform should support initially two training types: free walking with custom weight relief variations and granular support for specific motion patterns (T2).
- **G3 Code for Logging**: The programs should facilitate therapists on keeping track of the machine settings on each session (T3).
- **G4 Code for Collaboration**: The programs should promote collaboration between physiotherapists (T3).
- **G5 Openness for External Data Visualization Clients**: Data from the robot sensors and control parameters should be available for implementation of custom clients from third parties (T5),
- **G6 Openness for External Data Sources**: The platform should be open for including external sensing modules in the programs (e.g. heart monitor, walking-error detection) (T4).
- **G7 Support for Setup and Activity Workflows**: Programs should be able to describe both the main training mode and the setup mode (i.e., pre-setting the robot, attaching end effector to harness, and hoisting the patient) (T6).

5 THE PROPOSED DOMAIN-SPECIFIC LANGUAGE

To address the design goals defined in the previous section, we proposed a EUD environment supported by a scenario-based DSL. Our work is inspired by a DSL designed to support consistent BDD scenarios [62, 65]. The adaptation of this DSL to our context is appropriate as it has been originally designed to support event-driven systems, which include robots. The DSL has been implemented using the Xtext [2] framework. Programs are designed in textual artifacts called *model* files comprising structural and logical sections [22]. The programming process includes *i)* defining the entities involved in the training activity and *ii)* expressing their interaction with the Gherkin syntax as a series of scenarios written in a semi-structured natural language format.

5.1 Structural Specification

Figure 3b presents a basic template used for defining the structural section of a model. A formal Backus-Naur Form (BNF) grammar description is shown in Figure 3a. The interactive elements

```
################ Model Declaration #################
<model>::='model' <ID>
          (<model_references>)*
          <structural_definition>
          <logical_definition>

<model_references>::='using' <ID>

############### Structural Definition ###############
<structural_definition>::=(<entity_definition>|<device_definition>)*
<entity_definition>::='entity' <STRING> ('is' <parent> (',' <parent>)*)?
                      ('{'
                      <entity_attributes>
                      '}')?

<device_definition>::='device' <STRING> ('is' <parent> (',' <parent>)*)?
                      ('{'
                      <entity_attributes>
                      ('initial' 'state:' <state_definition>)?
                      '}')?

<entity_attributes>::=('actions:'<action_definition>(','<action_definition>)*)?
                       ('states:'<state_definition>(','<state_definition>)*)?
                       ('properties:'<property_definition>(','<property_definition>)
*)?

<action_definition>::= <STRING> (((['<modifier>'])?('the' <argument>)?)|
                            ('the' <STRING>)?(['<modifier>'])?)?
<state_definition>::= <STRING> ['<modifier>']?
<property_definition>::= <parent>::= <modifier> ::=<argument>::=<STRING>
```

(a) Grammar specification of a model's structure.

```
model <Model Name>

entity <Interaction Element Name> {
    states: <List of States>
    actions: <List of Actions>
    properties: <List of Properties>
}

device <Device Element> {
    states: <List of States>
    actions: <List of Actions>
    properties: <List of Properties>
    initial state: <State Name>
}
```

(b) Basic template of a model's structural definition.

Fig. 3. Structural part of a model.

involved in a training activity are declared in this section of the code. These elements are divided into entities and devices. Through the entities the user can declare external elements that will interact with the system to change its behavior (e.g., the patient's hand). The developer has no power to program these items, so they are limited to evaluating their current actions and states. Devices, on the other hand, are reactive elements within the system, whose responses to inputs generated by entities can be programmed in the scenarios. Both elements share the *states*, *actions*, and *properties* attributes. However, devices also have the *initial state* attribute, which allows to define the device's first behavior when launching the program. Devices and entities can be imported and extended

in new model files. Therefore, an end-user can import the model provided by the engineer for a particular technology and declare sub-definitions of these elements. Declaring sub-entities and sub-devices is the typical process of specializing the domain model for a particular task and allows the end-user to use their lexical preferred terms for a particular model, simplifying the process of understanding and sharing their solutions (**G1**, **G3**).

It is possible to give more specificity to the attributes using modifiers. These are prepositions and adverbs surrounded by square brackets, declared next to an action or a state (e.g., move[up], move[down], pulling [softly]). The mapping between such lexical terms and their numerical sensor values is performed through a JSON file at the user's working directory. The user can experiment with different forces and positions, and register new labels; or work with numeric values directly into the code, e.g., typing *with 5 kg* instead of *strongly* (**G1 G3**).

Listing 1 presents a basic model for the RoboTrainer. The *body_part* entity allows to define a body part relevant for the particular training activity, such as the hand for an arm curl, the hips for a core training, or the knees for sit-to-stand repetitions. These can be in states such as *holding*, *moving*, *laying*, or *pulling*; and perform actions such as moving to a particular position or pulling the rope with a particular force.

```
model robotrainer_domain

entity body_part {
    states:  holding, moving, laying, pulling
    actions: move[up], move[down], pull
    properties: position, force
}

device robot {
    states: pulling [softly]
    properties: force, position, velocity
    initial state: pulling [softly]
}
```

Listing 1. Base specification of the RoboTrainer domain. Users can import this model and use sub-entities of these elements in their programs.

The RoboTrainer is declared as a *robot* device that can be in a pulling state with different forces. The available properties are its *force*, *position*, and *velocity*. If not specified otherwise in a sub-declaration, its initial state is to pull *softly* (mapped to one kilogram-force), thus providing a quick out-of-the-box-pull mode for placing the end effector into a patient's harness or testing different positions for a movement (**G7**).

5.2 Logical Specification

The logical section of the code allows the user to write scenarios to specify the set of desired robot behaviors based on inputs from the entities. Each scenario represents state machine transitions with three main sections: *i)* one or more initial states for the elements (*"Given"*), *ii)* one or more conditions involving the elements (*"When"*), and *iii)* one or more resulting states for the elements after the conditions are satisfied (*"Then"*). In this way, each completed scenario generates a state change in the robot's behavior during the execution. Users can create one or more independent scenarios within the same program file to initiate a single-scenario passive unloading activity or to engage in active assistance/resistance training with sequential, discontinuous force responses.(**G1**, **G2**).

```
################## Logical Definiton #################
<logical_definition>::=(<scenario>)*
<scenario>::='Scenario:' <STRING>
             'Given' <scenario_state>
             'And' (<scenario_state>)*
             'When' <scenario_action>
             'And' (<scenario_action>)*
             'Then' <scenario_state>
             'And' (<scenario_state>)*

<scenario_state>::= <pre_post_words> <state_phrase>
<state_phrase>::=(<element_reference> <verb_to_be>?<state_name>
   (<modifier>? <preposition>? ('the'? '"'<property_value>'"' property_reference)?)
  |(<preposition>? ('the'? '"'<property_value>'"' property_reference)?<modifier>?))

<scenario_action>::=<when_words><element_reference><actions>
<actions>::= <action_reference><adverbial_clause>?
<adverbial_clause>::=<direction_clause>?<mode_clause>?
<direction_clause>::=('to'|'towards') 'the'? <property_value> <property_reference>?
<mode_clause>::='with' ('the'|'a')? <property_value> <property_reference>
<element_reference>::=<STRING><element_id>
<action_reference>::= <STRING> (((<modifier>)?('the' <STRING>)?)|
                          ('the' <STRING>)?(<modifier>)?)?

<when_words>::=('I'|'the')?('do' 'not'|'does' 'not')?
<verb_to_be>::=('is'|'are')('not')
<pre_post_words>::=('the'|'all' 'the'|'I')?('do' 'not'|'does' 'not')?
<preposition>::=('of'| 'in'| 'from'|'on'| 'to'| 'into'| 'for'| 'with'| 'up'|
'down'| 'out'| 'off'| 'at')
<element_id>::=('#' <INT>|'"'<STRING>'"')?
<property_value>::= '"'<STRING>'"'
<entity_name>::=<property_reference>::=<state_name>::=<modifier>::= <STRING>
```

(a) Grammar specification of the model's logical section.

```
1 Scenario: <Name>
2 Given the <User's Entity> is <Initial State>
3 And the <Device> is <Initial State>
4 When the <User's Entity> <Action>
5 Then the <Device> is <Resulting State>
```

(b) Basic template used for defining a RoboTrainer scenario.

Fig. 4. Logical part of a model.

Figure 4b presents the basic syntax that composes each scenario for a RoboTrainer task, while Figure 4a shows the BNF formal grammar description of such scenarios. The logical and structural grammar definitions were implemented in Xtext to generate the metamodel illustrated in Figure 5.

5.3 Interpretation and Execution

To interpret the scenarios, Behave [4] executes all the *Given*, *When*, and *Then* clauses as sequential steps. Each step is mapped to an implementation function in Python by using annotations with regular expressions. Traditionally, Behave's methods where designed for a linear test execution workflow rather than a recurrent activity. It was, therefore, necessary to modify its workflow to adapt to the execution of a continuous control loop managing the state machine of the training robot. For this purpose, the source code of Behave was modified so that tests were performed recurrently, and all tests reporting capabilities were suppressed to reduce their potential performance overhead. The new execution logic adopted is presented in Figure 6. Such an approach allows to evaluate

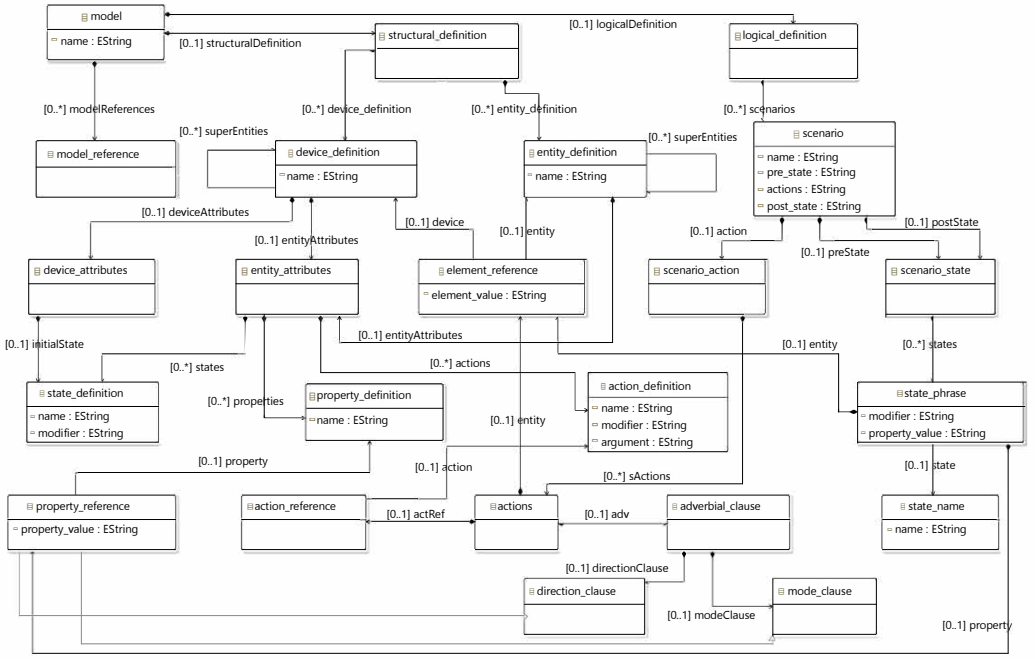


Fig. 5. The DSL metamodel.

all the scenarios sequentially in a continuous loop. A positive evaluation of the *Given* and *When* clauses of one of the scenarios results in a change of state in the device.

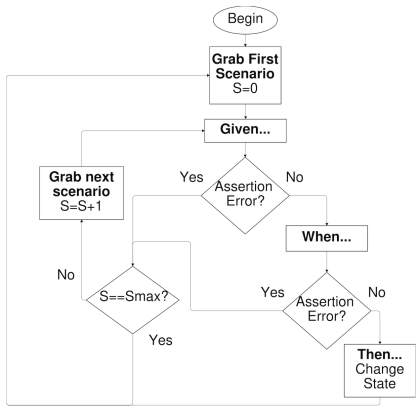


Fig. 6. Behave's execution logic was modified to loop through all the scenarios continuously, in order to act as a controller for the state machines built by the user ([23]).

Figure 7 shows an example of how an implementation function is mapped to a sample *Given* phase: *Given the hand is at the "top" position.* Here, the program will poll the rope length from the robot and validate if it is within the ranges labeled as "top" position in the environment's JSON file. If the validation is positive, Behave will continue with the next step of the scenario. On the

other hand, if a validation is negative, an assertion error would terminate the scenario without advancing to the following steps, and Behave will then begin with the next scenario available.

```
Given the hand is at the "top" position
@given('{article:Article} {entity} {state} {prep:Prep} {position}')
@given('{article:Article} {entity} {state} {prep:Prep} {position} position')
def step_impl(context,article,entity,state,prep,position):
    rt_pos=rt_model.getRobotPosition()
    upper_limit=env.get_position(position)[UPPER]
    lower_limit=env.get_position(position)[LOWER]
    assert (rt_pos<=upper_limit and rt_pos>=lower_limit)
```

Fig. 7. Implementation of a "Given" clause using the Behave framework.

6 SYSTEM CONFIGURATION AND COMPONENTS

Fig. 8 presents an overview of the system's components based on four hardware elements: A PC (IDE client), a Raspberry Pi (Device Server), a WiFi router, and an RoboTrainer unit. The user's code is developed on the IDE client. Once completed, it is deployed to the device server, which interprets the user scenarios and translates them into control parameters for updating the RoboTrainer's response. In return, the RoboTrainer provides the device server with current readings from its sensors, which are translated into information about the user's application state and sent back to the IDE client for visualization.

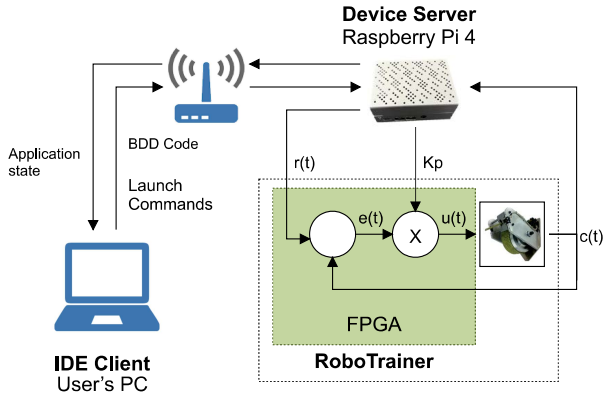


Fig. 8. Overview of the main system components: A PC (IDE client), a Raspberry Pi (Device Server), a WiFi router, and an RoboTrainer unit.

6.1 RoboTrainer

RoboTrainer is a wire-pulling device actuated by a 3-phase brushless DC motor. It uses an FPGA to modulate the motor's voltage through a proportional controller [68]. The controller's set points (r), proportional control gains (K_p), control signal (u), error (e), and output (y) are modeled by the following expressions:

$$y = \begin{bmatrix} z \\ \dot{z} \\ V \\ i \\ F \end{bmatrix} \qquad r = \begin{bmatrix} z_r \\ \dot{z_r} \\ V_r \\ i_r \\ F_r \end{bmatrix}$$

$$K_p = \left[K_{pz}, K_{p\dot{z}}, K_{pV}, K_{pi}, K_{pF}\right] \qquad e = r - y \qquad u = eK_p$$

Where z is rope length, $\dot{z}$ rope velocity, V voltage, i current, and F force. This hybrid control configuration allows for varying the combination of setpoints and control parameters to obtain responses adapted to the user's requirements. The response signal is associated by the user to a particular robot behavior [69]. For example, a combination with dominant Kp_z parameters will give a springy sensation around the target position (z_r) or even the perception of a static rope if K_p is high enough. On the other hand, using Kp_F provides a pulling behavior, allowing the user to train with a force close to the target F_r.

These behaviors can be modified during runtime, since the K_p, r, and y values used by the FPGA reside in a shared memory space of the RoboTrainer. A PC can use the Unity Link USB bridge protocol [46] to read the robot state and overwrite the control parameters, thus generating a change in the controller's response. Therefore, the proportional controller in the FPGA allows real-time control of the rope, while the high-level execution of the domain expert's logic is delegated to the device server.

6.2 Client IDE

Previous work introduced Platypus [24] as a development environment for robot-assisted physical training. It is comprised by a set of Eclipse IDE plug-ins, which provide utilities for managing robots, authoring routines, deploying code, and executing the programs remotely. For this work, such tools were adapted to integrate with the text-editor-support tools from Xtext. This provides the user with support through all the development and execution cycle through the following interactive elements:

6.2.1 Device View. This view includes actions to manage the RoboTrainer at different stages of the development flow (**G7**). The interaction between the IDE and the Device Server is done through the SSH protocol for launching programs and SFTP for deploying feature files:

- **Add/Remove a device**: When pressed, a connection wizard will guide the user to enter the credentials and test the connection with a remote device server.
- **Deploy a program**: When pressed, a deploy wizard will guide the user to select a BDD project that s/he wishes to deploy on the RoboTrainer. Platypus will then use regular expressions to clean up the user's program to make it compliant with the Behave syntax rules and display it as a feature file on the Device Server.
- **Execute/Stop**: Platypus will remotely launch/terminate a Behave node that executes the scenarios deployed by the user.

6.2.2 Editor View. Model files terminated with a .bdd extension are opened with the Xtext-generated editor, which provides the user with syntax highlights, error detection, and code hints. Additionally, upon execution of the code, Platypus establishes a TCP socket communication with the device server for monitoring the active states and actions. On every change, Platypus updates the text editor with colored annotations indicating the current state and actions (Figure 12).

6.3 Device Server

Figure 9 presents a simplified view of the execution environment implemented. A blackboard architecture [19] was chosen, whose components are distributed between the IDE client and the device server. In this way, different independent nodes (i.e., knowledge sources) work together on a common data structure hosted by the device server, while they are governed by a controller in the IDE client whose execution strategy depends on the user's actions and the state of the robot.

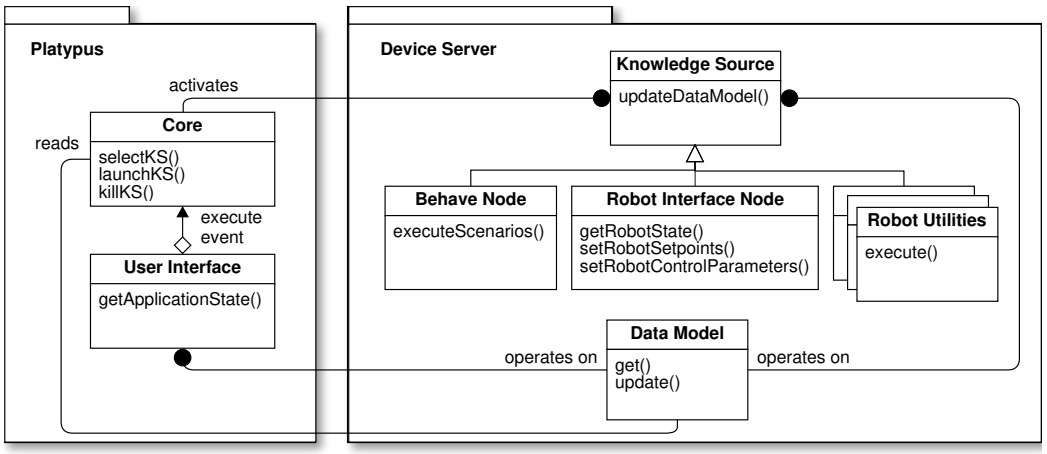


Fig. 9. Class diagram presenting the main components of the execution environment.

6.3.1 Robot Interface Node. This node interfaces with the robot using the Unity Link protocol for performing reading and writing tasks on its shared memory. The reading task involves polling the robot's state at a constant frequency and updating it in the data model, thus allowing other nodes to work with the most current state of the robot. The interface monitors the model for the existence of new setpoints $r(t)$ or control parameters K_p published by other nodes. If there are any, the write task consists of writing these values into the robot's memory to update its response.

6.3.2 Robot Utilities. Utility nodes are enabled by the IDE's controller when pre-defined routines such as enabling the robot or calibrating the force and position sensors are necessary. For this, the controller reads the robot's state from the data model and launches any calibrations needed before running the code (**G7**).

6.3.3 Behave Node. When the user chooses to launch their code, the controller launches the Behave node to interpret the user's code and begin the execution. By implementing sub-classes of custom Robot Interface nodes, third parties can plug in custom data clients and knowledge sources, thus providing an open platform for plugging custom clients and data sources to the running application (**G5**, **G6**).

7 USER WORKFLOW

To illustrate the workflow of our solution, we present the scenario where a user (i.e., the physical trainer) creates a simple routine for an abdominal pulldown exercise, in which the robot's resistance is higher during the eccentric phase of the motion than during the concentric one. Figure 10 presents an overview of this activity, illustrating an angular motion ω around the trainee's hips and the eccentric and concentric forces ($F_{concentric}$, $F_{eccentric}$) exerted by the robot on each phase.

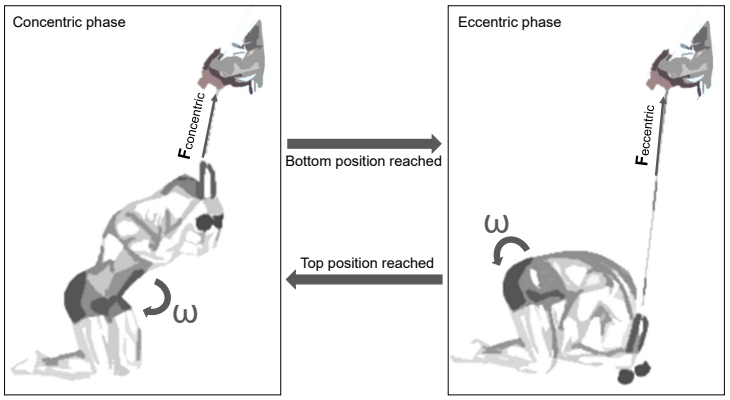


Fig. 10. Overview of the abdominal pulldown exercise assisted by a rope pulling device, where different support levels are provided by the device for the eccentric and concentric phases of movement.

7.1 Robot Connection and Setup

The robot is turned on. The user opens the IDE and uses the *Device View* to connect remotely. After entering the robot's address and login credentials into the wizard, the user can validate that the connection is successful. As the robot was just turned on, the user is prompted with a homing wizard. Instructions are given to visually validate that the robot is ready to be activated and has no motor obstructions. Thereafter, the robot performs the homing routine to calibrate the position and force sensors. Finally, the environment is ready to begin the programming process.

7.2 Ideation and Exploration

User and trainee discuss the exercise together (Figure 11a). After the training objectives and dynamics are clear to the trainee, the user creates a BDD project and programs the robot to pull softly (i.e., using the *softly* modifier, mapped by the JSON file to 1 kg-force). The code for a soft, constant pulling force is deployed and executed.

An exploratory phase begins, where the user and the trainee engage in physical demonstrations with the rope pulling at a constant force (Figure 11b). While the trainee becomes familiar with the dynamics of the exercise, the user maintains continuous communication with them, iteratively modifying the resistance until reaching a magnitude pleasing both trainer and trainee. During these explorations, the user also labels a feature that will define the change of phases in the training activity. For the abdominal pulldowns, the user identifies that the hips are roughly at 33 degrees when the concentric phase starts, and 100 degrees when the phase finishes and the eccentric phase begins. Therefore, the top position is labeled *33_degrees* and the bottom *100_degrees*. This is stored in a JSON file that relates the user's labels with the corresponding robot's position sensor readings.

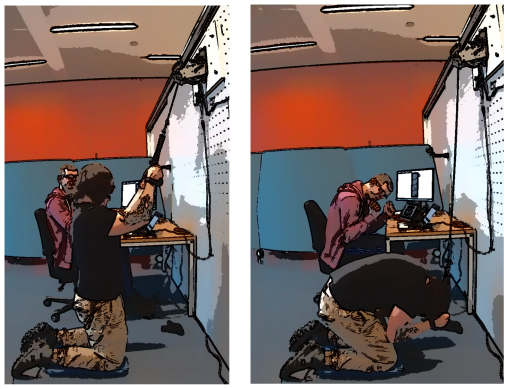
7.3 Coding the Solution

Figure 12 presents a final DSL code solution for the task. Since the hips are used for describing a change of phase, these are defined in the code as *body_part* entities. One scenario is written for each phase (i.e., eccentric and concentric). Each of them follows a similar structure:

a) In the *Given* section, the user specifies the initial condition of the trainee and the robot.
b) In the *When* section, the user defines an action from the trainee that will trigger a change of behavior in the robot.



(a) Ideation.



(b) Trainer-Trainee exploration.

Fig. 11. Steps of the user workflow.

```
model AbsPulldown
using robotrainer_domain

device robotrainer is robot{
    initial state: pulling with "5 kg"
}
entity hips is body_part

Scenario:"Concentric phase"
Given the hips are at the "33_degrees" position
And the robotrainer is pulling with "5 kg"
When the hips move down to the "100_degrees" position
Then the robotrainer is pulling with "10 kg"

Scenario:"Eccentric phase"
Given the hips are at the "100_degrees" position
And the robotrainer is pulling with "10 kg"
When the hips move up to the "33_degrees" position
Then the robotrainer is pulling with "5 kg"
```

Fig. 12. Program describing an abdominal crunch exercise at runtime. The lines highlighted in green color indicate current states of the system. The lines in yellow indicate actions taking place.

c) The *Then* section specifies the new behavior from the robot after the action is completed.

7.4 Launching the Solution

When the new program is deployed and launched, the IDE will highlight the user's scenarios with the current state of the application. For example, Figure 12 highlights a state where *a)* the trainee's hips are at the *33_degrees* position, *b)* the robot is pulling with a 5 kilogram force magnitude, and *c)* the trainee is instantly moving down towards the bottom position (*100_degrees*). Such a feedback allows the user to be aware of the robot's conditions at runtime, therefore being able to identify the current phase of the activity, the actions being performed by the robot, and the possibilities of triggering a different behavior of the device from the given situation.

8 QUALITATIVE EVALUATION

We conducted an evaluation workshop with a domain expert in order to explore the feasibility of our approach. The user, a physical therapist doing their PhD at our university, reported a period of 2-5 years of clinical experience and an entry-level with no formal education in computer programming (less than two months of self-paced Python learning from web forums). The participant was introduced to the authoring environment and tasked to program a training routine that maximized the feedback force provided by the robot in the middle segment of the range of motion of a single-arm pull-down exercise. Finally, a semi-structured interview was performed after the programming activity to obtain qualitative data about the user's experience.

Based on the recordings gathered during the interview, we have performed an axial comparison to understand how the use of our application relates with the practices of therapists on the clinical setting (i.e., index case):

- **Data Extraction**: The data was anonymized, cleaned and transcribed. Notes where taken to associate each section of the transcript to the recording times.
- **Code data**: Concepts of interest were identified and categorized in a systematic fashion throughout the transcript.
- **Axial Categorization**: Groups of codes were related to the themes from the index case.

8.1 Results

Fig. 13 reveals the distribution of the identified codes within the categories of the index case. The distribution highlights the participant's focus on discussing their experience in terms of how language coherence aligns with their clinical competencies (T1), the impact of language on their time management with technology (T6), and the significance of the chosen model regarding the essential elements of defining an exercise from their perspective (T4). Such observations are discussed bellow.

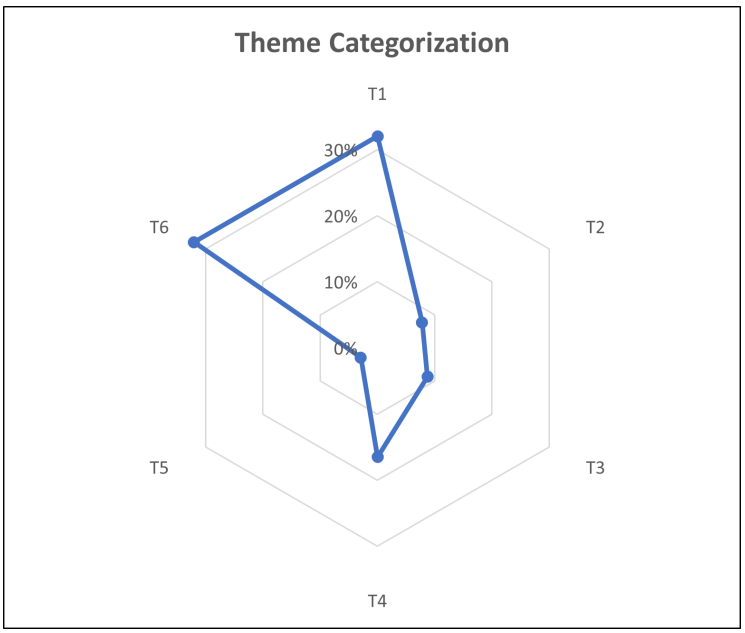


Fig. 13. Theme categorization of the post-workshop interview data.

8.1.1 Preserving and Sharing Knowledge. Although being able to solve the task without any issues, the participant acknowledged a significant effort when trying to relate the task requirements with the abstraction level provided by the language. While his initial focus was to solve the task using 3 scenarios, one for each phase of the motion, he realized that with the schema provided, the solution required 4 different scenarios: one for each phase transition. From his perspective, this contrasts sharply with a traditional viewpoint, wherein task-related repetitive movements are characterized by distinct phases within the range of motion, rather than the transitions between these phases. Fundamentally, the key distinction observed between these perspectives lies in the emphasis: the transition-focused approach highlights the cyclical nature of movement, necessitating definitions for the changes in force during eccentric and concentric movements. Conversely, the phase-oriented perspective presumes that the range of motion remains constant in both directions of movement (**C18**).

> **C18** *"If you look at the picture there is a soft-hard-soft phase. But I actually have to make a soft-hard-soft-and-hard phase in order to make the program function. Because it has to loop over it (...). A clinician won't think like that. They will make one for each phase and try to make that work. And they won't succeed because they won't have the last part that cycles back through the hard part to get to the first soft part."*

8.1.2 Key Points During Therapy. The change of focus on the level of abstraction towards phases within the range of motion also involves improvements of parameters provided to the user. Among them:

(1) The use of ranges, specified in percentages of the rope length to delimit the phases of a movement (**C19**).
(2) The optional use of the standard keywords "eccentrically" and "concentrically" to optionally allow granularity to be preserved in the direction of movement (**C20**).

> **C19** *"If I had the percentage of movement then I would probably use that. Like 0%-100%.. Then if I wanted to define the middle part, that would be intuitive for me. Oh, the middle part is starting at 40% and ending in 70%. That would be intuitive. But writing −0.3 to −0.5 [is not]."*
> **C20** *"You could make a function that splits in concentric, eccentric. That is a standard procedure."*

8.1.3 Time Negotiations. Consistent with T6, the participant highlights the need to reduce program configuration times by enhancing the code re-usability through practices such as:

(1) A soft-calibration interface that allows recording the top and bottom positions of a range of motion through physical demonstrations of the top and bottom positions.
(2) The use of code snippets for auto-insertion of the Given... When... Then... template, or conversely the adoption of a visual-based approach to drag and drop scenario structures (**C21**).
(3) The use of drop-down widgets to allow the user to select among the declared entities (**C22**).

C21 *"So these blocks are always the same right? Scenario, Given, And, When, Then? So why not make a program that contains these blocks?"*

C22 *"The only entity that exists in the system is the shoulder, right?! So always in "Given" the only option is shoulder. But there is no need for me to write that. It could just be a drop-down menu that actually contains shoulders or just be automatically input."*

9 IMPLICATIONS

The application of our BDD-based DSL illuminates the potential scalability of our language within the wider context of bodily human-robot interaction through a framework that allows scenarios to be executed recursively. On a broader context, the proposed solution shows a potential to be adapted to both robotic and non-robotic systems that require continuous execution of scenarios. Particularly, some themes identified through the analysis show an alignment with the theoretical principles of the diffusion of innovations in service organizations ([29], [63]) describing some key elements for the adoption of innovations in the health sector. Specifically, themes T1 and T2 resonate with the theory's assertions about complexity in service organizations; Rogers defines this concept as 'the degree to which an organization's members possess a relatively high level of knowledge and expertise' [63], suggesting that complexity in organizations challenges the adoption of innovations by making it difficult for adopters to reach consensus on the implementation of the innovation. Particularly, clinical organizations are usually characterized by a high level of complexity from practitioners with a wide variety of highly specialized knowledge (e.g., nurses, medics, therapists, etc.). Through the use of BDD, our language allows the user to use a semi-structured natural language with which they feel comfortable to define the behavior of the technology they use. Its use within the domain of healthcare organizations has the potential to alleviate the complexity barriers of implementing new innovations.

10 LIMITATIONS

Safety against injury is traditionally achieved by limiting the combination of force, velocity and acceleration to non-harmful thresholds. While the physical limits inherent to the robot itself will prevent injury, there's still a risk factor derived from programs causing unexpected behaviors due to programming errors. Confusing and scaring people for instance by sudden stops, jerks or changes in force, as we often experience during experiments, would create annoying or potentially dangerous situations, such as patients tripping or loosing equilibrium. Future work in this dimension involves identifying strategies to assist the user during the development of the program to prevent unexpected behaviours. To address this issue, recent literature proposes several strategies. These include for example providing contextual hints to enhance the user's learning experience (e.g., [35]), facilitating group development activities with human peers or AI agents (e.g., [38], [61], [36]), or implementing automated code synthesis-and-repair solutions presented to the end-users (e.g., [30, 31]).

Furthermore, the active involvement of the patient during robot-assisted training sessions has not been studied in this work. There is a need to complement the therapists-centered design by exploring how the patient's participation could influence the programming language through a new patient-centered participatory analysis.

Our solution enhances collaboration between therapists by facilitating the exchange of ideas through concrete, written scenarios. This allows users to work together in specifying a task or to adapt pre-existing programs to their own objectives. However, our work does not include a software infrastructure to enable online collaboration among users. This represents a significant

opportunity for future development, potentially drawing inspiration from successful low-code tools in the market, such as Node-RED [1], where users can share and download code units or entire execution flows through an online community library. Nonetheless, there are specific challenges related to user privacy, especially since our design aims to enable users to track a patient's progress through code (**G3**). This highlights an overlap between the use of programs as tools for logging (**G3**) and for collaboration (**G4**). The adoption of an online work environment, although necessary to enhance the system's flexibility and collaboration between practitioners, requires the adoption of strategies that guarantee the basic security aspects of cyberphysical systems [52]: confidentiality, integrity, and availability of information.

Further limitations not addressed by our solution on its current state include the following aspects:

(1) Our DSL does not yet support the use of the robot's position control mode despite it being a relevant feature according to theme (T4). Currently the only configurable states are based on force control. In future versions, the DSL will take advantago of the RoboTrainer's hybrid controller for allowing the specification of static (i.e., position-control based) and dynamic (force-control based) working modes.
(2) The text editor does not validate the sequential correctness of the user's programs. Currently, the Xtext-based editor validates the syntactic correctness of the developed code, however, the user is not certain of the logical validity of the program, therefore allowing to specify unreachable states. Future work will take advantage of the capabilities of the Eclipse RCP framework to perform logical validations of the user's code before the application's deployment. Furthermore, the work by Kang et al. [37] shows promising developments in the mapping process between BDD scenario files and formal computation models, thus offering a potential enhancement for our solution.
(3) Smooth transition between states is not guaranteed. Although our solution has the strength of allowing the elaboration of discontinuous feedback sequences by the robot, in its current phase it does not guarantee a smooth transition between the states specified by the user. We recognize the need to adopt intelligent policies such as safety shielding [41] to both improve the user experience, and protect the technology from unexpected power spikes.

11 CONCLUSIONS AND FUTURE WORK

This work has allowed us to obtain two products in parallel. On the one hand, after two phases of field interviews, we have achieved a synthesis of key insights to guide our design of a language for robot-assisted exercising based on design goals. On the other hand, using a development-based requirement acquisition, we have a functional solution that currently simplifies the development time of behaviours needed for both clinical and human-robot-interaction validations relying on cable-based training devices.

Among its main features, our solution presents a comprehensive end-user development environment that allows the user to *i)* write programs using a written BDD based DSL, *ii)* calibrate and configure devices for an activity through Eclipse's RCP capabilities, *iii)* deploy and run their code, and *iv)* obtain visual feedback during the execution.

Through the use of BDD, the developed code aligns with clinical requirements by facilitating collaboration and progress tracking, while simultaneously producing functional, executable artifacts. The DSL covers both structural and logical dimensions of a training activity, thus allowing the domain expert to *i)* define the entities involved in training using their vocabulary, and *ii)* define the robot's behaviour as a set of interactive scenarios in the Gherkin syntax. For the execution of the

[1] https://nodered.org/

scenarios developed by the user, a customized version of Behave was optimized for the recurring execution of the state machine controlling the robot. Furthermore, we presented an overview of the hardware and software components of our implementation, which is comprised of a development client, a training robot, and a device server translating the user's logic into the robot's lower control specifications.

Based on observations from both rounds of interviews, we propose as future work the possibility of specifying movement by basing the scenarios on phases, rather than transitions. A couple of sample proposals can be seen in Figs. 14-15, compared to the current solutions for some shoulder and abdominal pull-down exercises, respectively. Fundamentally, this would enable the user to use a trigger-action based approach without a conception of the underlying state machine structure, being the "Given" sentence an optional contextual element differentiating particular components of the motion (i.e., concentric, eccentric). This provides a new balance between the granularity of the language for a diverse range of applications (T1) and the time invested in the technology, both in the process of solving the problem and in the time saved writing the scenarios (T6).

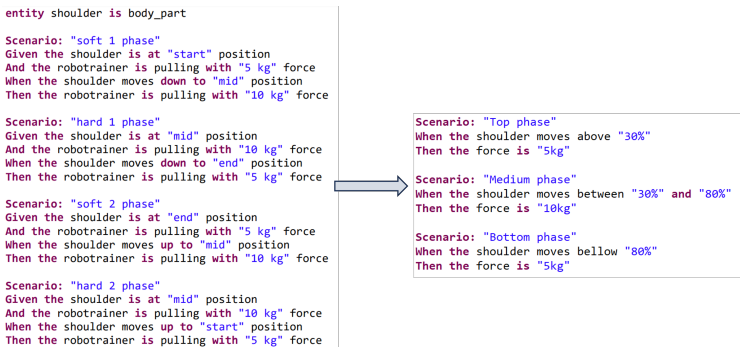


Fig. 14. Example of a new model applied on a multi-phased exercise with no restrictions on the movement direction.

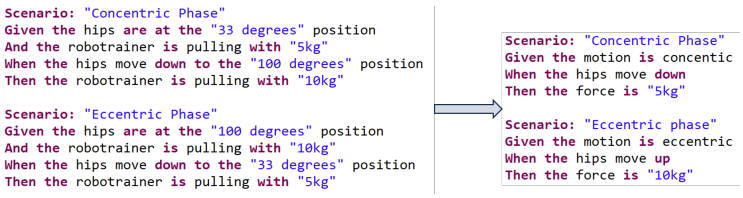


Fig. 15. Example of a new model applied on an exercise with restrictions on the movement direction.

While the focus of this work has started on a textual approach to highlight the key language-specific requirements, future work will also focus on time optimizations of the authoring process through the adoption of hybrid programming methods. As a first step, we plan to implement our BDD approach into previous solutions utilizing block-based programming interfaces ([24]). Furthermore, a demonstration-based interface is planned to assist the therapist into guiding the choice of boundaries for each phase within the range of motion. Finally, model-wise, we plan to take advantage of the scalability from the architecture proposed to explore how the adoption of key elements described by T5, such as *challenge* and *rope-behavior* could affect the language.

Subsequently, a second round of tests will be carried out with a larger pool of physiotherapists at different levels of expertise. In this case, the users will be evaluated in their total authorship of exercises without the existence of a base activity. This study will allow us to investigate their ability to map their own defined requirements to the actual definition of entities and scenarios involved in the structural and logical specification of the program.

ACKNOWLEDGMENTS

This work was supported by the University of Southern Denmark's (SDU) Internal Strategic Fund. We are deeply grateful to our fellow researchers at SDU's Tinity Lab[2] for creating an enriching atmosphere for collaboration and creativity. We thank Anders Holsgaard, Morten Lynggaard Pedersen, Jon Skovgaard Jensen, Per Aagaard and Jakob Nielsen for their invaluable insights from the health science and sports fields, Gitte Rasmussen and Emilie Munch Steffensen for enriching our understanding from the humanities perspective, Mads Bergholdt Sørensen for his engineering expertise, and Ulrik Pagh Schultz Lundquist for his expert guidance and supervision. Finally, special thanks to Lourdes Rebeca Samaniego for providing us with insightful feedback and a fresh perspective, and to Manuel Sebastian Parrao Alcantara for his outstanding mentorship in software engineering.

REFERENCES

[1] 2022. LiteGait. https://www.litegait.com/product/lg-500x. Accessed: 2024-02-13.
[2] 2023. *Xtext*. https://www.eclipse.org/Xtext/ Accessed: 10th of May, 2023.
[3] 2024. Armeo Spring. https://www.hocoma.com/solutions/armeo-spring/. Accessed: 2024-02-13.
[4] 2024. *behave*. https://behave.readthedocs.io/en/latest/ Accessed: 16 Feb 2024.
[5] 2024. *Cucumber*. https://cucumber.io Accessed: 16 Feb 2024.
[6] 2024. Ergo Trainer. https://www.winncare.uk/fiche-produits-rehabilitation-ergo_trainer-12-630-en.htm. Accessed: 2024-02-13.
[7] 2024. *Gherkin*. https://cucumber.io/docs/gherkin Accessed: 10 May 2024.
[8] 2024. *JBehave*. https://jbehave.org Accessed: 16 Feb 2024.
[9] 2024. Lokomat. https://www.hocoma.com/solutions/lokomat/. Accessed: 2024-02-13.
[10] Gojko Adzic. 2011. *Specification by example: how successful teams deliver the right software*. Simon and Schuster.
[11] Gopika Ajaykumar, Maureen Steele, and Chien-Ming Huang. 2021. A Survey on End-User Robot Programming. *ACM Comput. Surv.* 54, 8, Article 164 (oct 2021), 36 pages. https://doi.org/10.1145/3466819
[12] Zaid Al-Saadi, Yahya M. Hamad, Yusuf Aydin, Ayse Kucukyilmaz, and Cagatay Basdogan. 2023. Resolving Conflicts During Human-Robot Co-Manipulation. In *Proceedings of the 2023 ACM/IEEE International Conference on Human-Robot Interaction* (Stockholm, Sweden) *(HRI '23)*. Association for Computing Machinery, New York, NY, USA, 243–251. https://doi.org/10.1145/3568162.3576969
[13] Barbara Rita Barricelli, Fabio Cassano, Daniela Fogli, and Antonio Piccinno. 2019. End-user development, end-user programming and end-user software engineering: A systematic mapping study. *Journal of Systems and Software* 149 (2019), 101–137.
[14] Angelo Basteris, Sharon M. Nijenhuis, Arno H. A. Stienen, Jaap H. Buurke, Gerdienke B. Prange, and Farshid Amirabdollahian. 2014. Training modalities in robot-mediated upper limb rehabilitation in stroke: a framework for classification based on a systematic review. *Journal of neuroengineering and rehabilitation* 11, 1 (2014), 111–111.
[15] A. Bellucci, A. Vianello, Y. Florack, L. Micallef, and G. Jacucci. 2019. Augmenting objects at home through programmable sensor tokens: A design journey. *International Journal of Human Computer Studies* 122 (2019), 211–231. https://doi.org/10.1016/j.ijhcs.2018.09.002
[16] Sara Beschi, Daniela Fogli, and Fabio Tampalini. 2019. CAPIRCI: A multi-modal system for collaborative robot programming. In *International Symposium on End User Development*. Springer, 51–66.
[17] Margaret Burnett, Curtis Cook, Omkar Pendse, Gregg Rothermel, Jay Summet, and Chris Wallace. 2003. End-user software engineering with assertions in the spreadsheet paradigm. In *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE, 93–103.
[18] Margaret Burnett, Andrei Sheretov, Bing Ren, and Gregg Rothermel. 2002. Testing homogeneous spreadsheet grids with the" what you see is what you test" methodology. *IEEE Transactions on Software Engineering* 28, 6 (2002), 576–594.

[2]https://trinity-motion.org/about-us

[19] Frank Buschmann, Michael Kircher, and Douglas C. Schmidt. 2008. *Pattern-oriented Software Architecture: A System of Pattern,*. Wiley.

[20] Kathy Charmaz. 2017. Constructivist grounded theory. *The journal of positive psychology* 12, 3 (2017), 299–300.

[21] Yuchen Cui, Siddharth Karamcheti, Raj Palleti, Nidhya Shivakumar, Percy Liang, and Dorsa Sadigh. 2023. No, to the Right: Online Language Corrections for Robotic Manipulation via Shared Autonomy. In *Proceedings of the 2023 ACM/IEEE International Conference on Human-Robot Interaction* (Stockholm, Sweden) *(HRI '23)*. Association for Computing Machinery, New York, NY, USA, 93–101. https://doi.org/10.1145/3568162.3578623

[22] Jose Pablo De La Rosa, Thiago Rocha Silva, and Anders Stengaard Sørensen. 2024. End-User Programming of Robot-Assisted Physical Training Activities through Behaviour-Driven Development. In *Companion of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*. 387–391.

[23] Jose Pablo De la Rosa, Anders Stengaard Sørensen, and Thiago Rocha Silva. 2023. Towards a Low-Code Programming Environment for Robot-Assisted Physical Training Activities. In *2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 287–289.

[24] Jose Pablo De la Rosa Gutierrez and Anders Stengaard Sørensen. 2023. Platypus: An environment for end-user development of robot-assisted physical training. In *Companion of the 2023 ACM/IEEE International Conference on Human-Robot Interaction*. 342–346.

[25] Yvonne Dittrich, Olle Lindeberg, and Lars Lundberg. [n. d.]. End-user development as adaptive maintenance. In *End user development*. Springer, 295–313.

[26] Martin Flück, Rebekka Bosshard, and Max Lungarella. 2017. Cardiovascular and Muscular Consequences of Work-Matched Interval-Type of Concentric and Eccentric Pedaling Exercise on a Soft Robot. *Frontiers in physiology* 8 (2017), 640–640.

[27] Martin Fowler. 2011. *Domain-specific languages*. Addison-Wesley, Upper Saddle River, NJ.

[28] Razan Ghzouli, Thorsten Berger, Einar Broch Johnsen, Swaib Dragule, and Andrzej Wąsowski. 2020. Behavior trees in action: a study of robotics applications. In *Proceedings of the 13th ACM SIGPLAN International Conference on Software Language Engineering* (Virtual, USA) *(SLE 2020)*. Association for Computing Machinery, New York, NY, USA, 196–209. https://doi.org/10.1145/3426425.3426942

[29] Trisha Greenhalgh, Glenn Robert, Fraser Macfarlane, Paul Bate, and Olivia Kyriakidou. 2004. Diffusion of Innovations in Service Organizations: Systematic Review and Recommendations. *The Milbank quarterly* 82, 4 (2004), 581–629.

[30] B. Hofer, R. Abreu, A. Perez, and F. Wotawa. [n. d.]. Generation of relevant spreadsheet repair candidates. In *Frontiers in Artificial Intelligence and Applications*, Vol. 263. 1027–1028. https://doi.org/10.3233/978-1-61499-419-0-1027

[31] B. Hofer and F. Wotawa. [n. d.]. Mutation-based spreadsheet debugging. In *2013 IEEE International Symposium on Software Reliability Engineering Workshops, ISSREW 2013*. 132–137. https://doi.org/10.1109/ISSREW.2013.6688892

[32] Gaoping Huang, Pawan S. Rao, Meng-Han Wu, Xun Qian, Shimon Y. Nof, Karthik Ramani, and Alexander J. Quinn. 2020. Vipo: Spatial-Visual Programming with Functions for Robot-IoT Workflows. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3313831.3376670

[33] Justin Huang and Maya Cakmak. 2017. Code3: A System for End-to-End Programming of Mobile Manipulator Robots for Novices and Experts. In *Proceedings of the 2017 ACM/IEEE International Conference on Human-Robot Interaction* (Vienna, Austria) *(HRI '17)*. Association for Computing Machinery, New York, NY, USA, 453–462. https://doi.org/10.1145/2909824.3020215

[34] IAIS, F. 2020. Open Roberta Lab. https://lab.open-roberta.org/.

[35] Will Jernigan, Amber Horvath, Michael Lee, Margaret Burnett, Taylor Cuilty, Sandeep Kuttal, Anicia Peters, Irwin Kwan, Faezeh Bahmani, and Amy Ko. 2015. A principled evaluation for a principled idea garden. In *2015 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 235–243. https://doi.org/10.1109/VLHCC.2015.7357222

[36] Le Jiang, Mohd Anuaruddin Bin Ahmadon, Di Zhang, Tomoya Fukada, and Shingo Yamaguchi. 2023. SynergyAI: An Human-AI Pair Programming Design Tool Based on Program Net. In *2023 11th International Conference on Information and Education Technology (ICIET)*. 210–214. https://doi.org/10.1109/ICIET56899.2023.10111108

[37] Eun-Young Kang and Thiago Rocha Silva. 2023. Towards Formal Verification of Behaviour-Driven Development Scenarios Using Timed Automata. In *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*. 612–616. https://doi.org/10.1109/APSEC60848.2023.00081

[38] F. Karlsson. [n. d.]. Using two heads in practice. In *Proceedings - International Conference on Software Engineering*. 43–47. https://doi.org/10.1145/1370847.1370857

[39] Moiz Khan, Tatiana Luna, Victor Santamaria, Isirame Omofuma, Dario Martelli, Enrico Rejc, Joel Stein, Susan Harkema, and Sunil Agrawal. 2019. Stand Trainer With Applied Forces at the Pelvis and Trunk: Response to Perturbations and Assist-As-Needed Support. *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 27, 9 (2019), 1855–1864. https://doi.org/10.1109/TNSRE.2019.2933381

[40] Amy J Ko, Robin Abraham, Laura Beckwith, Alan Blackwell, Margaret Burnett, Martin Erwig, Chris Scaffidi, Joseph Lawrance, Henry Lieberman, Brad Myers, et al. 2011. The state of the art in end-user software engineering. *ACM Computing Surveys (CSUR)* 43, 3 (2011), 1–44.

[41] Bettina Könighofer, Florian Lorber, Nils Jansen, and Roderick Bloem. 2020. Shield Synthesis for Reinforcement Learning. In *Leveraging Applications of Formal Methods, Verification and Validation: Verification Principles*, Tiziana Margaria and Bernhard Steffen (Eds.). Springer International Publishing, Cham, 290–306.

[42] T. Kosar, Z. Lu, M. Mernik, M. Horvat, and M. Črepinšek. 2021. A case study on the design and implementation of a platform for hand rehabilitation. *Applied Sciences (Switzerland)* 11, 1 (2021), 1–21. https://doi.org/10.3390/app11010389

[43] Tomaž Kosar, Zhenli Lu, Marjan Mernik, Marjan Horvat, and Matej Črepinšek. 2021. A Case Study on the Design and Implementation of a Platform for Hand Rehabilitation. *Applied Sciences* 11, 1 (2021). https://doi.org/10.3390/app11010389

[44] Alyssa Kubota, Emma I. C. Peterson, Vaishali Rajendren, Hadas Kress-Gazit, and Laurel D. Riek. 2020. JESSIE: Synthesizing Social Robot Behaviors for Personalized Neurorehabilitation and Beyond. In *Proceedings of the 2020 ACM/IEEE International Conference on Human-Robot Interaction* (Cambridge, United Kingdom) *(HRI '20)*. Association for Computing Machinery, New York, NY, USA, 121–130. https://doi.org/10.1145/3319502.3374836

[45] H. Lamine, S. Bennour, and L. Romdhane. 2015;2016;. *Dynamic Simulation of a Cable-Based Gait Training Machine*. Mechanisms and Machine Science, Vol. 37. Springer International Publishing, Cham, 199–207.

[46] Anders Blaabjerg Lange, Ulrik Pagh Schultz, and Anders Stengaard Soerensen. 2013. Unity-link: A software-gateware interface for rapid prototyping of experimental robot controllers on FPGAs. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. 3899–3906. https://doi.org/10.1109/IROS.2013.6696914

[47] Nicola Leonardi, Marco Manca, Fabio Paternò, Carmen Santoro, and Assoc Comp Machinery. 2019. Trigger-action programming for personalising humanoid robot behaviour. Assoc Computing Machinery, NEW YORK.

[48] Henry Lieberman, Fabio Paternò, and Volker Wulf. 2006. *End User Development*. Vol. 9. Springer Science & Business Media.

[49] T. Lourens. 2004. TiViPE - Tino's Visual Programming Environment. In *Proceedings of the 28th Annual International Computer Software and Applications Conference, 2004. COMPSAC 2004.* 10–15 vol.1. https://doi.org/10.1109/CMPSAC.2004.1342799

[50] Ying Mao and Sunil Kumar Agrawal. 2012. Design of a Cable-Driven Arm Exoskeleton (CAREX) for Neural Rehabilitation. *IEEE Transactions on Robotics* 28, 4 (2012), 922–931. https://doi.org/10.1109/TRO.2012.2189496

[51] Laura Marchal-Crespo, Lars Michels, Lukas Jaeger, Jorge López-Olóriz, and Robert Riener. 2017. Effect of Error Augmentation on Brain Activation and Motor Learning of a Complex Locomotor Task. *Frontiers in neuroscience* 11 (2017), 526–526.

[52] Peter Marwedel. 2018. *Introduction*. Springer International Publishing, Cham, 1–25. https://doi.org/10.1007/978-3-319-56045-8_1

[53] C. Mayr-Dorn, M. Winterer, C. Salomon, D. Hohensinger, and R. Ramler. 2021. Considerations for using Block-Based Languages for Industrial Robot Programming - A Case Study. In *Proceedings - 2021 IEEE/ACM 3rd International Workshop on Robotics Software Engineering, RoSE 2021*. 5–12. https://doi.org/10.1109/RoSE52553.2021.00008

[54] G. Ng, J. G. Shin, A. Plopski, C. Sandor, and D. Saakes. [n. d.]. Situated game level editing in augmented reality. In *TEI 2018 - Proceedings of the 12th International Conference on Tangible, Embedded, and Embodied Interaction*, Vol. 2018-January. 409–418. https://doi.org/10.1145/3173225.3173230

[55] Dan North. 2006. Introducing BDD. *Better Software* (2006).

[56] Dan North. 2022. What's in a Story? http://dannorth.net/whats-in-a-story/

[57] Fabio Paternò. 2020. *End-User Development*. Springer International Publishing, Cham, 1–27. https://doi.org/10.1007/978-3-319-27648-9_37-1

[58] Fabio Paternò and Volker Wulf. 2017. *New perspectives in end-user development*. Springer.

[59] W. C. Payne, Y. Bergner, M. E. West, C. Charp, R. B. Shapiro, D. A. Szafr, E. V. Taylor, and K. DesPortes. 2021. Danceon: Culturally responsive creative computing. In *Conference on Human Factors in Computing Systems - Proceedings*. https://doi.org/10.1145/3411764.3445149

[60] E. Pot, J. Monceaux, R. Gelin, and B. Maisonnier. 2009. Choregraphe: a graphical tool for humanoid robot programming. In *RO-MAN 2009 - The 18th IEEE International Symposium on Robot and Human Interactive Communication*. 46–51. https://doi.org/10.1109/ROMAN.2009.5326209

[61] A. Repenning, N. Ahmadi, N. Repenning, A. Ioannidou, D. Webb, and K. Marshall. [n. d.]. Collective programming: Making end-user programming (more) social. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, Vol. 6654 LNCS. 325–330. https://doi.org/10.1007/978-3-642-21530-8_34

[62] Thiago Rocha Silva. 2022. Towards a Domain-Specific Language to Specify Interaction Scenarios for Web-Based Graphical User Interfaces. In *Companion of the 2022 ACM SIGCHI Symposium on Engineering Interactive Computing*

Systems (Sophia Antipolis, France) *(EICS '22 Companion)*. Association for Computing Machinery, New York, NY, USA, 48–53. https://doi.org/10.1145/3531706.3536463

[63] Everett M. Rogers. 2003. *Diffusion of innovations* (5 ed.). Free Press, New York.

[64] Muhammad Shoaib, Ehsan Asadi, Joono Cheong, and Alireza Bab-Hadiashar. 2021. Cable Driven Rehabilitation Robots: Comparison of Applications and Control Strategies. *IEEE Access* 9 (2021), 110396–110420. https://doi.org/10.1109/ACCESS.2021.3102107

[65] Thiago Rocha Silva. 2023. Towards a Domain-Specific Language for Behaviour-Driven Development. In *2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 283–286. https://doi.org/10.1109/VL-HCC57772.2023.00054

[66] Jon Skovgaard Jensen, Anders Holsgaard-Larsen, Anders Stengaard Sørensen, Per Aagaard, and Jens Bojsen-Møller. 2024. Acute effects of robot-assisted body weight unloading on biomechanical movement patterns during overground walking. *Journal of Biomechanics* 162 (2024), 111862. https://doi.org/10.1016/j.jbiomech.2023.111862

[67] Sruti Subramanian, Yngve Dahl, Beatrix Vereijken, and Dag Svanæs. 2021. ExerTiles: A Tangible Interactive Physiotherapy Toolkit for Balance Training with Older Adults. In *Proceedings of the 32nd Australian Conference on Human-Computer Interaction* (Sydney, NSW, Australia) *(OzCHI '20)*. Association for Computing Machinery, New York, NY, USA, 233–244. https://doi.org/10.1145/3441000.3441043

[68] Anders S. Sørensen, Jacob Nielsen, Jørgen Maagaard, Martin Skriver, Chih-Cheng Lin, and Ulrik P. Schultz. 2016. Low-cost modular robotic system for neurological rehabilitative training. In *2016 IEEE International Conference on Industrial Technology (ICIT)*. 1585–1591. https://doi.org/10.1109/ICIT.2016.7474997

[69] Anders S. Sørensen and Gitte Rasmussen. 2018. RoBody Interaction A New Approach at Kinesthetic Human Robot Interaction. IEEE. https://doi.org/10.1109/roman.2018.8525844

[70] Daniel Tetteroo, Paul Vreugdenhil, Ivor Grisel, Marc Michielsen, Els Kuppens, Diana Vanmulken, and Panos Markopoulos. 2015. Lessons Learnt from Deploying an End-User Development Platform for Physical Rehabilitation. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* (Seoul, Republic of Korea) *(CHI '15)*. Association for Computing Machinery, New York, NY, USA, 4133–4142. https://doi.org/10.1145/2702123.2702504

[71] M. Verano Merino and T. Van Der Storm. 2020. Block-based syntax from context-free grammars. In *SLE 2020 - Proceedings of the 13th ACM SIGPLAN International Conference on Software Language Engineering, Co-located with SPLASH 2020*. 283–295. https://doi.org/10.1145/3426425.3426948

First Submitted July 2023; Revised February 2024; Revised April 2024; Accepted April 2024